LAB 5: STATIC CODE ANALYSIS

Telecom Tower Maintenance Tracker

Name: Salasha Vijay SRN: PES1UG24CS689 Section: L

1. Summary of Results

Tool	Before	After
Pylint	5.14/10	10.00/10
Bandit	B105 hardcoded password; B608 possible SQL injection; B301 pickle use	No issues identified
Flake8	E501 long lines; E303 excess blank lines; E225 operator whitespace	No violations reported (empty report)

The final program output was ['Ridge-A'] high 2 1. The final Pylint score was 10.00/10, Bandit reported “No issues identified,” and Flake8 produced an empty report, as shown in the terminal screenshots.

2. Issue Documentation Table

Line numbers refer to the original file as shown in the initial tool reports; line numbers changed when the program was rewritten.

Issue / tool code	Type	Original line(s)	Description	Fix approach / outcome
Hardcoded password (B105)	Security	6	Password-like value 'tower-noc-77' was stored in source code and could expose a credential if shared.	Removed the unused NOC_PASSWORD constant; it was not used by the program.
Possible SQL injection (B608)	Security	11	The INSERT query was built with string formatting rather than binding the input values.	Changed to SQLite placeholders (?, ?, ?) and passed the values as a tuple.
Unsafe pickle use (B301)	Security	43	snapshot() used pickle.loads(pickle.dumps(...)) unnecessarily; pickle deserialization can be unsafe with untrusted data.	Removed pickle and returned fetched SQLite rows directly.
Long lines (E501 / C0301)	Style / maintainability	5, 11, 18, 35, 43	Several source lines exceeded the 79-character limit.	Split SQL statements and expressions across lines; final Flake8 report was empty.
Excess blank lines (E303)	Style	25	Too many consecutive blank lines appeared between functions.	Removed excess blank lines.
Missing operator whitespace (E225)	Style	50	total=len(snapshot()) lacked spaces around the assignment operator.	Changed to total = len(snapshot()).
Missing docstrings (C0114 / C0116)	Code quality	1, 10, 15, 25, 34, 42, 46	The module and functions did not consistently explain their purpose.	Added module and function docstrings.
Non-snake_case names (C0103)	Maintainability	10, 15, 34	Names such as addTower, serviceTower, and riskLevel did not follow Python naming conventions.	Renamed functions to add_tower, service_tower, and risk_level, updating call sites.
Broad exception handling (W0718)	Reliability	20	Catching Exception could hide unrelated errors and complicate debugging.	Changed the handler to catch sqlite3.Error.
Global statement (W0603)	Maintainability	16	The maintenance function modified a module-level counter with global.	service_tower() now returns success/failure; main() tracks service_count locally.
Missing-row handling	Robustness improvement	riskLevel function	The original function accessed row[0] without checking whether a row existed.	Added a None check and return 'unknown' for a nonexistent tower ID.

3. Reflection Questions and Answers

The provided handout says to answer the reflection questions but does not show the exact questions. These sample answers align with the handout's learning outcomes; replace the wording if your instructor supplied a separate question sheet.

Q1. What is static code analysis, and how did it help in this lab?

Static code analysis examines source code without executing it to find potential bugs, security risks, and style problems. Pylint highlighted maintainability and documentation issues, Bandit identified unsafe patterns, and Flake8 found formatting violations. Running the tools before and after the changes provided evidence that the issues were addressed.

Q2. What was the most important security issue identified?

The original program built an SQL INSERT statement using string formatting, creating a possible SQL injection risk. Parameterized queries separate SQL code from supplied values. Bandit also flagged a hardcoded password-like string and pickle use; the unused string and unnecessary pickle operations were removed.

Q3. How did Pylint and Flake8 differ?

Pylint checked broader code quality, including naming, docstrings, broad exception handling, and global statements. Flake8 focused on style and formatting, including line length, blank lines, and whitespace. Their feedback was complementary.

Q4. Why should broad exception handling be avoided?

Catching Exception can hide errors the code did not intend to handle and make debugging harder. Catching sqlite3.Error limits handling to database-related failures and makes the behavior clearer.

Q5. How were the fixes verified?

I ran the program and confirmed the output remained ['Ridge-A'] high 2 1. I reran Pylint, Bandit, and Flake8. The final Pylint score was 10.00/10, Bandit reported no issues, and Flake8 produced no findings. I also saved before-and-after reports and screenshots.

Q6. What did you learn about code quality and maintainability?

Code can run correctly while still containing security, readability, or style problems. Clear names, docstrings, parameterized SQL, specific exception handling, and consistent formatting make code easier to review and maintain. Automated tools help, but their findings must be interpreted in context.

4. Verification Evidence

- Original Pylint score: 5.14/10.
- Final Pylint score: 10.00/10.
- Final Bandit result: No issues identified.
- Final Flake8 report: empty; no style violations were printed.
- Final program output: ['Ridge-A'] high 2 1.
- Before-and-after report files: pylint_before.txt, pylint_after.txt, bandit_before.txt, bandit_after.txt, flake8_before.txt, flake8_after.txt.
- Git commits visible on the fork: eb2705b, 332f61f, and 4beab38.

Cleaned and Updated tower_maintenance.py

```
"""Manage tower maintenance records using an in-memory SQLite database."""

import sqlite3


conn = sqlite3.connect(":memory:")
conn.execute(
    """
    CREATE TABLE towers (
        id INTEGER PRIMARY KEY,
        site TEXT,
        last_service INTEGER,
        faults INTEGER
    )
    """
```

```
)


def add_tower(site, last_service, faults=0):
    """Add a tower to the database."""
    conn.execute(
        """
        INSERT INTO towers (site, last_service, faults)
        VALUES (?, ?, ?)
        """,
        (site, last_service, faults),
    )
    conn.commit()


def service_tower(tower_id, year):
    """Record tower maintenance and reset its fault count."""
    try:
        cursor = conn.execute(
            """
            UPDATE towers
            SET last_service = ?, faults = 0
            WHERE id = ?
            """,
            (year, tower_id),
        )
        conn.commit()

        if cursor.rowcount > 0:
            return True

        print("Service failed: tower not found.")
        return False

    except sqlite3.Error as error:
        print(f"Service failed: {error}")
        return False


def overdue(year):
    """Return tower sites whose last service was at least two years ago."""
    rows = conn.execute(
        "SELECT site, last_service FROM towers"
    ).fetchall()

    return [
        site
        for site, last_service in rows
        if year - last_service >= 2
    ]


def risk_level(tower_id):
    """Return the risk level based on the tower's fault count."""
    row = conn.execute(
        "SELECT faults FROM towers WHERE id = ?",
        (tower_id,),
    ).fetchone()

    if row is None:
        return "unknown"

    if row[0] > 3:
        return "high"

    return "normal"


def snapshot():
    """Return a snapshot of all tower records."""
    return conn.execute("SELECT * FROM towers").fetchall()


def main():
    """Run a demonstration of the tower maintenance functions."""
```

```
    add_tower("Ridge-A", 2021, 5)
    add_tower("Coast-B", 2024)

    service_count = 0
    if service_tower(2, 2025):
        service_count += 1

    total = len(snapshot())
    print(overdue(2025), risk_level(1), total, service_count)


if __name__ == "__main__":
    main()
```